Lecture 9. Pipes

Alternatives to the pipe, when not to use it, other magrittr tools

Contents

9.1 Why pipes **2**

9.2 When not to use the pipe **3**

9.3 Other tools from magrittr **3**

9.4 Selected exercises **3**

9.1 Why pipes

Motivation. This lecture starts the programming part of the course. Code is a means of communication, with the computer and with the people (including your future self) who will read it. The pipe is the first tool for writing code that reads in the same order as the analysis it performs.

The pipe `%>%` comes from the `magrittr` package; tidyverse packages load it automatically. To see why it helps, write the same sequence of actions four ways, using the nursery rhyme "Little bunny Foo Foo went hopping through the forest, scooping up the field mice and bopping them on the head".

Save each intermediate step.

```
foo_foo <- little_bunny()
foo_foo_1 <- hop(foo_foo, through = forest)
foo_foo_2 <- scoop(foo_foo_1, up = field_mice)
foo_foo_3 <- bop(foo_foo_2, on = head)
```

The code is cluttered with unimportant names, and the suffixes must be incremented carefully (it is easy to use `foo_foo_1` where `foo_foo_2` was meant). Worry about memory is usually misplaced: R shares columns between modified copies of a data frame, so copies are cheap until a column changes.

Overwrite the original.

```
foo_foo <- hop(foo_foo, through = forest)
foo_foo <- scoop(foo_foo, up = field_mice)
foo_foo <- bop(foo_foo, on = head)
```

Debugging is painful (a mistake means rerunning from the start), and the repeated name hides what changes on each line.

Function composition.

```
bop(scoop(hop(foo_foo, through = forest), up = field_mice), on = head)
```

The code must be read inside out and right to left, and each function's arguments end up far from its name (the "Dagwood sandwich" problem).

The pipe.

```
foo_foo %>%
  hop(through = forest) %>%
  scoop(up = field_mice) %>%
  bop(on = head)
```

The code reads as a sequence of verbs, "Foo Foo hops, then scoops, then bops". Behind the scenes magrittr rewrites it as a function with a placeholder `.`, roughly

```
my_pipe <- function(.) {
  . <- hop(., through = forest)
  . <- scoop(., up = field_mice)
  bop(., on = head)
}
```

Consequently the pipe does not work with functions that use the current environment (`assign()`, `get()`, `load()`) or rely on lazy evaluation in a special way (`tryCatch()`, `try()`, `suppressMessages()`).

The native pipe [Supplement]

Since R 4.1 base R has its own pipe `|>`, which works like `%>%` in simple cases: `x |> f(y)` is `f(x, y)`. It is slightly faster and needs no package, but it has no `.` placeholder in the same form (R 4.2 added `_` for named arguments) and none of the special pipes below. These notes

use `%>%`, as the course does.

9.2 When not to use the pipe

Pipes are for a linear sequence of operations on one object. Use something else when

- the pipe is longer than about ten steps: create intermediate objects with meaningful names, which also makes debugging easier;
- there are several inputs or outputs, i.e. no single primary object is being transformed;
- the steps form a directed graph with complex dependencies rather than a chain.

9.3 Other tools from magrittr

The tee pipe `%T>%` returns its left-hand side instead of the right-hand side, so a step with a side effect (plotting, printing, saving) does not end the pipeline. Without it, `plot()` returns `NULL` and the next step receives nothing:

```
rnorm(100) %>%
  matrix(ncol = 2) %>%
  plot() %>%
  str()
```

```
##  NULL
```

```
rnorm(100) %>%
  matrix(ncol = 2) %T>%
  plot() %>%
  str()
```

```
##  num [1:50, 1:2] -0.387 -0.785 -1.057 -0.796 -1.756 ...
```

The exposition pipe `%$%` "explodes" the variables of a data frame so they can be referred to by name in functions without a `data` argument:

```
mtcars %$% cor(disp, mpg)
```

```
## [1] -0.8475514
```

The assignment pipe `%<>%` replaces `mtcars <- mtcars %>% transform(cyl = cyl * 2)` by `mtcars %<>% transform(cyl = cyl * 2)`. Many prefer to avoid it: assignment is a special operation that should be clearly visible at the start of a line.

9.4 Selected exercises

Motivation. The pipe is defined by a rewriting rule. The exercises derive what the rule implies: how long chains and the dot placeholder are translated, why `assign()` misbehaves inside a pipe, why some functions need the exposition pipe, and how the native pipe differs.

Exercise 9.1 (the rewriting rule)

Take as definition that `x %>% f(y)` means `f(x, y)` when the dot `.` does not appear as an argument of `f`, and means `f(y)` with `.` replaced by `x` when it does.

(a) Prove that a chain `x %>% f1(y1) %>% ⋯ %>% fk(yk)` without dots evaluates to $f_k(\cdots f_2(f_1(x, y_1), y_2) \cdots, y_k)$.
(b) Predict, then check, the values of `1:3 %>% c(4, .)` and `1:3 %>% c(length(.))`.

Solution [Supplement].

(a)

Step 1. The pipe is left-associative: the chain is read as `((x %>% f1(y1)) %>% f2(y2)) %>% ⋯`. Induct on k. For $k = 1$ the claim is the definition.

Step 2. If the chain of length $k-1$ evaluates to $v = f_{k-1}(\cdots f_1(x, y_1) \cdots, y_{k-1})$, then the chain of length k pipes the value v into `fk(yk)`, which by definition is $f_k(v, y_k)$. This is the claim for k.

(b) In `c(4, .)` the dot is an argument, so the result is `c(4, 1:3)`. In `c(length(.))` the dot appears only inside the nested call `length(.)`, not as an argument of `c`; the left-hand side is therefore still inserted first, giving `c(1:3, length(1:3))`:

```
1:3 %>% c(4, .)
```

```
## [1] 4 1 2 3
```

```
1:3 %>% c(length(.))
```

```
## [1] 1 2 3 3
```

■

Exercise 9.2 (assignment inside a pipe)

Explain the following output. Why does `assign()` inside a pipe not create a variable unless the environment is given explicitly?

```
"zz" %>% assign(100)
exists("zz")
```

```
## [1] FALSE
```

```
env <- environment()
"zz" %>% assign(100, envir = env)
exists("zz")
```

```
## [1] TRUE
```

Solution [Supplement].

Step 1. `assign(x, value)` writes into the environment in which it is evaluated (its default `pos = -1` means "the current environment").

Step 2. The pipe does not paste `assign("zz", 100)` into the calling code. As the lecture's sketch `my_pipe` shows, it evaluates the right-hand side inside a function-like temporary environment in which `.` is bound. The variable `zz` is created there and disappears when the pipe returns, so `exists("zz")` is `FALSE`.

Step 3. Capturing the calling environment first and passing it as `envir` tells `assign()` where to write, so the second call creates `zz` where we can see it. The same applies to `get()` and `load()`: functions that act on "the current environment" see the pipe's environment, not ours. ■

Exercise 9.3 (when the exposition pipe is needed)

Why does the first call below fail while the second works?

```
mtcars %>% sum(cyl)
## Error: object 'cyl' not found

mtcars %$% sum(cyl)
## [1] 198
```

Solution [Supplement]. By the rule of Exercise 9.1 the first call is `sum(mtcars, cyl)`. The pipe supplies the data frame as an argument, but `cyl` is still looked up as an ordinary variable in the calling environment, where it does not exist. Functions such as `lm()` avoid this with a `data` argument; `sum()` has none. The exposition pipe evaluates `sum(cyl)` in an environment in which the columns of `mtcars` are variables, much like `with(mtcars, sum(cyl))`, so `cyl` is found and the total number of cylinders, 198, is returned. ■

Exercise 9.4 (the native pipe is syntax)

Explain the following outputs, and why `1:3 |> sum` is an error while `1:3 %>% sum` works.

```
quote(x |> f(y))
## f(x, y)

quote(x %>% f(y))
## x %>% f(y)

tryCatch(str2lang("1:3 |> sum"), error = conditionMessage)
## [1] "The pipe operator requires a function call as RHS (<input>:1:8)"
```

Solution [Supplement].

Step 1. `quote()` returns an expression without evaluating it. The native pipe is rewritten by the parser: `x |> f(y)` *is* the call `f(x, y)` before anything runs, so no pipe function is ever called and there is no overhead.

Step 2. The magrittr pipe is an ordinary function `` `%>%` `` with two arguments, so the quoted expression stays a call to that function; the rewriting happens only when it is evaluated.

Step 3. Since the parser must insert `x` as the first argument of a call, the right-hand side of `|>` has to be written as a call, `sum()`. The magrittr function, by contrast, receives the unevaluated right-hand side and accepts a bare function name, turning `1:3 %>% sum` into `sum(1:3)`. ■